

Fault Mitigation Patterns

Introduction to a collection of secure programming patterns for security-critical devices

Abstract

Hardware fault attacks can break software security by revealing secrets during program execution or a change of the behavior of a program. Without profound knowledge of these attacks, it is hard to defend code effectively. Fault resistance requires pervasive protection throughout the code. This paper introduces a collection of secure programming patterns for security-critical devices. These patterns help developers mitigate the risk of fault injection in a cost-effective way.

Author: Marc Witteman

Introduction

Hardware fault injection is a class of hardware security attacks which have become increasingly popular, as these attacks are powerful and have a high probability of success. Most devices today are completely vulnerable against these attacks as developers have little awareness of the threat and do not know how to protect their code. The software security implications of these attacks are discussed in [1].

Fortunately, it is possible to harden the software and mitigate the fault injection threat. However, since the number of attacker opportunities is very high, this could require a large effort. In this paper, we propose a set of 11 fault mitigation patterns. These patterns are advantageous because they can be repeatedly applied, without making a detailed design for each instance, and thus minimize the mitigation effort. We organize the patterns along with three main strategies:

1. **Resist:** Increase code resistance so that faults are less likely to disturb program behavior.
2. **Recover:** Code resilience to prevent insecure behavior following a fault.
3. **Respond:** Actions to deter attackers after detecting a fault.

The remainder of this paper details the patterns which implement these strategies. We include illustrative examples in the C language (translation to other languages be straightforward). These examples serve to clarify the pattern and can be adapted and applied for instances of the problem.

It is our experience that these patterns, if well understood, are applied efficiently at the average cost of 12 minutes per instance. Furthermore, there is typically no need to protect the entire code base: protecting the critical code is often good enough.

Resist

An attacker using fault injection aims for a specific effect, such as changing a value or decision. Resistance is introduced by making useful values hard to achieve, or by making it hard to find the right timing to manipulate a decision.

Pattern 1. FAULT.CONSTANT.CODING

Problem

Sensitive data carrying a limited set of values (like phase and state variables) is manipulated by fault injection attacks if they use trivial constant coding (for example, 0, 1, 0xFF, or -1).

Solution

Do not use trivial constants for sensitive data. These constants should use non-trivial values with the maximal hamming distance that is unlikely to be set through fault injection. This includes avoiding booleans which are coded in a trivial way (0 / 1). Use complex symbolic constants instead.

Example constant coding

```
int INITIALIZED = 0x5A5A;
```

```
int AUTHENTICATED = 0x3EE3;
```

Pattern 2. FAULT.RANDOM.DELAY

Problem

Attackers aim to manipulate a specific value or decision.

Solution

Insert random-length delays throughout the code, making it much harder to hit a specific moment in software execution.

Example random delay

```
void delay () { // wait random time

    int loops = rand() & 0x3FF; // 10 bit entropy, 1024 possible values

    while (--loops >= 0 ) {

        loops++;
        loops--; // loop counter changes avoid compiler removal of non-functional loop
    }
}
```

Recover

Even when fault injection is successful, it is possible to make resilient code that would continue correct execution and prevent exploitation. This includes double-checking to verify value correctness and crypto results, but also double-checking conditional statements, branches, loops, and program flow.

Pattern 3. FAULT.VALUE.CHECK

Problem

Sensitive data is manipulated by a fault injection attack at any time during program execution.

Solution

Verify sensitive data. Sensitive data can, for instance, be protected by a checksum. Data protected in this way should be verified at regular intervals. Ideally, the integrity of sensitive data should be verified at each use.

Example value check

```
int result = SOME_VALUE; // sensitive value assigned

int checksum = ~ SOME_VALUE; // use complement as checksum

if (checksum == ~result) { // verify checksum

    // continue critical code execution

}
```

Pattern 4. FAULT.CRYPTO.CHECK

Problem

Cryptographic algorithms are sensitive to fault injection. They may even reveal key data through the output of false encryptions due to fault injection. Differential fault analysis (DFA) is a technique for this issue.

Solution

Check for fault injection during or after crypto. Verify any ciphered data before transmission by deciphering or repeated enciphering. If the deciphered data matches the original input, or the repeated enciphering matches the original output, it is most likely that the encryption was not corrupted.

Example crypto check

```
encrypt( source, destination ); // result is in destination

decrypt( destination, source_copy ); // source_copy should be same as source

if (memcmp( source, source_copy, length) == 0) { // verify decryption

    send( destination ); // continue critical code execution

}
```

Pattern 5. FAULT.DECISION.CHECK

Problem

Sensitive data is manipulated by fault injection attacks. When a decision is made upon a single test, the decision may be corrupted.

Solution

Double-check sensitive conditions. A conditional process based on sensitive data should double-check the data. Preferably, these checks should not be identical, but complementary, as the attacker will have to perform two different types of attack. In addition, the checks should be temporally separated to reduce the risk of a single multi-cycle glitch bypasses check and double-check.

Example decision check

```
if (condition == SUCCESS) { // enter critical path

    int complement = ~SUCCESS;

    if (~condition == complement) { // check complement

        // continue critical code execution

    }

}
```

Pattern 6. FAULT.BRANCH.CHECK

Problem

Fault injection attacks manipulate branch choices.

Solution

Verify each branch. Use a different value for verification. Do not use default cases for valid behavior.

Example branch check

```
int inv_state = ~state; // create bitwise complement for verification

switch (state) {

    case INIT:

        if (inv_state == ~INIT) { // verify inv_state

            // continue critical code execution

        } break;

    case OPS:

        if (inv_state == ~OPS) {

            // verify inv_state // continue critical code execution

        } break;

}
```

Pattern 7. FAULT.LOOP.CHECK

Problem

Repetitive processes running in a loop are terminated early by a fault injection attack.

Solution

Verify loop completion, avoid early or late completion.

Example loop check

```
int i;

for ( i = 0; i < n; i++ ) { // important loop that must be completed

}

if (i == n) { // loop completed
```

```
// continue critical code execution  
  
}
```

Pattern 8. FAULT.NESTED.CHECK

Problem

Function calls may be skipped by a fault and render all included checks useless.

Solution

Verification and protected functionality should be at the same nest level.

Example Nested Check

```
void weak_protection() { // bad implementation  
  
    if (check_double() == TRUE) { // check_double can be skipped with single fault  
  
        // continue execution  
  
    }  
  
}  
  
int check_double() {  
  
    If (check_single() == TRUE && check_single() == TRUE) return TRUE;  
  
    return FALSE;  
  
}  
  
void better_protection() { // good implementation  
  
    if (check_single() == TRUE) {  
  
        if (~check_single() == ~TRUE) { // double check enforced  
  
            // continue critical code execution  
  
        }  
  
    }  
  
}
```



```

        }

    }

}

int check_single() {

    If (check == TRUE) return TRUE; // check has important value

    return FALSE;

}

```

Pattern 9. FAULT.FLOW.CONTROL

Problem

Fault injection can hit the program counter or stack. This can result in “code hijacking,” or unauthorized jumps to privileged code. Verification of the correct flow should be done during the execution of sensitive code.

Solution

Use a counter to keep track of the correctness of the execution path. Check the counter to verify the completion of the execution path. Prevent separate paths reaching identical counter values by stepping with prime numbers.

Example flow control

```

#define METHOD1_STEP 13

#define METHOD2_STEP 17

int counter;

void main( int argc, char** argv ) {

    counter = 0;

    method1( ... ); // call nested method

```

```

        if (counter == 2*METHOD1_STEP + METHOD2_STEP) { // check flow counter

            // continue critical code execution, knowing the full path was taken

        }
    }

    void method1( ... ) {

        counter += METHOD1_STEP; // increase counter

        method2( ... );

        counter += METHOD1_STEP; // increase counter

    }

    void method2( ... ) {

        counter += METHOD2_STEP; // increase counter

        // do other stuff

    }

```

Respond

Immunity is challenging, especially when so many vulnerabilities exist. It is therefore important to detect fault attempts and act accordingly.

Pattern 10. FAULT.DETECT

Problem

Repeated fault attempts may eventually be successful.

Solution

Test occurrence of logically impossible behavior to discover fault attempts. Include randomly occurring traps, deliberate verifications, or known values, to complicate avoidance of detection.

Example fault detect

```
#define SUCCESS = 0x3CA5C35A

#define INV_ SUCCESS 0xC35A3CA5

if (conditionalValue == SUCCESS) { // enter critical path

    trap(); // enter random delay where attack can be detected

    if (~conditionalValue == INV_ SUCCESS) { // check complement

        // continue critical code execution

    } else fault_detected(); // fault detected

}

void trap () { // wait random time and catch faults

    int loops = rand() & 0x3FF; // 10 bit entropy, 1024 possible values

    while (--loops >= 0 ) {

        if (~SUCCESS != INV_ SUCCESS) fault_detect(); // fault detected!

    }

}

void fault_detected() { // what to do when a fault is detected

    exit(); // terminate program

}
```

Pattern 11. FAULT.PENALTY

Problem

Immunity against fault injection is difficult, especially given the widespread nature of the problem. Attackers will therefore be inclined to repeat fault experiments and keep trying to find an exploitable weakness.

Solution

By the introduction of a penalty, it is possible to deter attackers and reduce their chance of success. Such a penalty can be in slowing down the system or disabling functionality. The disabling functionality can be temporary (for example, require service center intervention), or permanent (termination or key zeroing).

Example fault penalty

```
#define SUCCESS = 0x3CA5C35A

#define INV_SUCCESS 0xC35A3CA5

int fault_penalty_delay = 0;

if (conditionalValue == SUCCESS) { // enter critical path

    penalty_delay(); // include delay if attack detected earlier

    if (~conditionalValue == INV_SUCCESS) { // check complement

        // continue critical code execution

    } else fault_detected(); // fault detected

}

void penalty_delay () { // wait random time and catch faults

    if (fault_penalty_delay <= 0) return; // no attack penalty needed

    fault_penalty_delay--; // decrement penalty waits

    sleep(10000); // sleep 10 seconds

}

void fault_detected() { // what to do when a fault is detected

    fault_penalty_delay = 10; // introduce long waiting time in next 10 sessions

    exit(); // terminate program

}
```

Conclusion

Fault injection is a growing threat to devices that need to be secure in the field. Since the amount of fault injection vulnerabilities in software is overwhelming, there is a need for a systematic mitigation approach. We propose to use fault mitigation patterns, a set of 11 countermeasures that can be applied throughout the code and require little adaptation for repeated application.

Reference

- [1] Bilgiday Yuce, Patrick Schaumont, Marc Witteman, Fault Attacks on Secure Embedded Software: Threats, Design and Evaluation, <https://arxiv.org/pdf/2003.10513.pdf>